PROGRAMACIÓN EN PYTHON · SESIÓN 3 · BLOQUE 2

Control de Excepciones y la Filosofía EAFP

Gestión idiomática de errores con try, except, else y finally. De las checked exceptions de Java al paradigma pythónico.

2º DAM · Módulo Optativo · Código defensivo y tolerancia a fallos

PARADIGMAS DE DISEÑO

El choque filosófico: LBYL vs EAFP

LBYL: Look Before You Leap

Típico de **Java** y C++. Comprueba condiciones defensivamente con if antes de ejecutar cualquier acción susceptible de fallar.

```
// Comprobación previa manual
 if (divisor ≠ 0) {
     resultado = total / divisor;
 } else {
     System.out.println("No divisible");
 }
```

EAFP: Easier to Ask Forgiveness

El estándar idiomático en **Python**: "Es más fácil pedir perdón que pedir permiso". Ejecuta la acción y atrapa limpiamente el error si ocurre.

```
# Intenta y captura si falla
 try:
     resultado = total / divisor
 except ZeroDivisionError:
     print("No divisible")
```

ANATOMÍA DEL MANEJO DE ERRORES

Estructura: try, except, else y finally

try

Contiene el bloque de código riesgoso que puede lanzar excepciones.

except TipoError as e

Se activa únicamente si surge el error especificado. Se puede capturar el objeto.

else (¡No existe en Java!)

Se ejecuta SOLO si el bloque try finalizó con éxito total sin lanzar excepciones.

finally

Se ejecuta SIEMPRE (haya o no excepción). Ideal para liberar recursos.

```
try:
     num = int(input("Edad: "))
     calculo = 100 / num
except ValueError as err:
     print("Debes ingresar un número entero.")
except ZeroDivisionError:
     print("La edad no puede ser 0.")
else:
     print(f"Cálculo correcto: {calculo:.2f}")
finally:
     print("Fin de la validación.")

err = “invalid literal for int() with base 10: 'veinte'”
```

CATÁLOGO DE ERRORES HABITUALES

Excepciones típicas en 2º de DAM

ValueError

El tipo es correcto pero el valor es inválido para la operación.

```
int("hola")
 # ValueError
```

IndexError

Acceso a una posición inexistente en una lista, tupla o cadena.

```
items = [1, 2]
 x = items[5] # IndexError
```

TypeError

Operación invocada sobre un objeto de tipo incompatible.

```
len(42) # int no tiene len()
 "A" + 5 # TypeError
```

BUENAS PRÁCTICAS · PEP 8

El peligro del except genérico desnudo

Antipatrón: except a secas

Un except: sin tipo captura TODO, incluyendo KeyboardInterrupt (Ctrl+C en terminal) o errores tipográficos en variables. Oculta bugs graves.

Patrón profesional: Captura específica

Captura solo los tipos de error previstos. Si necesitas agruparlos en una sola línea, usa tuplas: except (ValueError, TypeError):.

```python
# MUY MAL: Oculta si el usuario pulsa Ctrl+C
try:
    procesar_archivo()
except:
    print("Algo ha fallado...")

# CORRECTO: Específico y con alias al error
try:
    procesar_archivo()
except (FileNotFoundError, PermissionError) as e:
    print(f"Error de acceso: {e}")
```

CONTROL ACTIVO DEL FLUJO

Lanzar excepciones: La sentencia raise

Equivalencia con el 'throw' de Java

En Java escribimos throw new IllegalArgumentException(...) y debemos declarar throws en la firma del método si es chequeada.

En Python usamos la palabra clave **raise**. En Python todas las excepciones son no chequeadas (no se declaran en firmas).

```python
def registrar_usuario(nombre, edad):
    if edad < 0:
        # Dispara una excepción con mensaje claro
        raise ValueError("La edad no puede ser negativa")

    if len(nombre) < 3:
        raise ValueError("El nombre es demasiado corto")

    return f"Usuario {nombre} registrado con éxito"
```

PATRÓN DE PRODUCCIÓN

El patrón estándar: Bucle de entrada robusta

¿Cómo pedir datos sin que la consola muera?

Combinar un bucle infinito while True con un bloque try-except-else garantiza que el programa jamás se caiga ante entradas inválidas y solo avance cuando el dato sea correcto.

El 'else' ejecuta el break solo si la conversión no falló.

```
while True:
    try:
        precio = float(input("Introduce precio (€): "))
        if precio ⩽ 0:
            raise ValueError("Debe ser mayor que 0")
    except ValueError as error:
        print(f"Entrada inválida ({error}). Reintenta.")
    else:
        # Solo llegamos aquí si no hubo ningún fallo
        break

print(f"Precio registrado: {precio:.2f} €")
```

TALLER 2 · MANOS AL TECLADO (20 MIN)

Práctica 2: Calculadora Segura y Tolerante a Fallos

Crear archivo: 08_calculadora_segura.py

1. Bucle interactivo: El script debe pedir **(input)** dos operandos y un operador (+, -, *, /) o la palabra SALIR.

2. Captura de errores: Manejar ValueError si introducen letras y ZeroDivisionError si intentan dividir entre 0.

3. Validación con raise: Si el operador no es uno de los cuatro válidos, lanzar un ValueError("Operador no soportado").

4. Cláusula else: El resultado debe imprimirse **estrictamente** dentro del bloque else del try.

EJEMPLO DE EJECUCIÓN ESPERADA:

```
Número 1: 15
 Operador (+, -, *, /): /
 Número 2: 0
[ERROR]: No se puede dividir entre cero.


 Número 1: 15
 Operador: +
 Número 2: 7
 [ÉXITO]: 15.0 + 7.0 = 22.00
```

LA SALIDA DEBE SER IGUAL QUE LA DEL EJEMPLO



¡Sesión 3 Completada!

Has dominado las secuencias con **Listas, Tuplas y Slicing** en el Bloque 1, y la robustez del código con **Excepciones EAFP** en el Bloque 2.

Próxima sesión: **Diccionarios (dict), Conjuntos (set) y Funciones modulares**.